



# CellTrack Web

快速开始指南 | v1.0

细胞分割与追踪系统 — 基于 YOLOv8 +  
DeepSORT

本指南帮助你在 15 分钟内完成环境搭建并启动应  
用

适用平台: Windows / macOS / Linux  
Python 3.8+ | Node.js  $\geq 20.19$  | MySQL



## 目录

---

**第0步** 开始前的准备 — 前置软件 & 项目目录切换

**第一步** 创建并激活 Python 虚拟环境 (Conda)

**第二步** 安装 MySQL 数据库

**第三步** 配置环境变量 (.env 文件)

**第四步** 安装 Python 依赖

**第五步** 初始化数据库

**第六步** 验证 ultralytics 本地库 (可选)

**第七步** 安装前端 npm 包

**启动** 启动前后端服务 + 故障排查 + 访问地址

 **提示：** 如果遇到问题，本指南末尾附带常见故障排查。

## 安装步骤一览：

| 步骤      | 做什么              | 关键命令                                                                                                 | 预计耗时  |
|---------|------------------|------------------------------------------------------------------------------------------------------|-------|
| ○       | 检查环境 + 进入项目目录    | <code>cd</code><br><code>/path/to/cell-yolo-web</code>                                               | 1 min |
| —       | 创建 conda 环境      | <code>conda create -n</code><br><code>cell-yolo</code><br><code>python=3.8</code>                    | 3 min |
| 二       | 安装 MySQL         | <code>brew install</code><br><code>mysql</code> 或下载安装包                                               | 5 min |
| 三       | 配置 .env          | <code>cp .env.example</code><br><code>.env</code> → 编辑密码                                             | 2 min |
| 四       | 安装 Python 包      | <code>pip install -r</code><br><code>requirements.txt</code>                                         | 5 min |
| 五       | 初始化数据库           | <code>python</code><br><code>init_db.py</code> →<br><code>migrate</code> →<br><code>init_data</code> | 2 min |
| 六<br>+七 | 验证本地库 + 安装<br>前端 | <code>npm install</code>                                                                             | 3 min |

## 0 开始前的准备 — 前置软件检查

在开始安装之前，请确认以下三个基础软件均已安装：

| 软件             | 用途     | 检查命令                          | 未安装?                                  |
|----------------|--------|-------------------------------|---------------------------------------|
| Python / Conda | 后端运行环境 | <code>python --version</code> | 安装 <a href="#">Miniconda</a>          |
| Node.js        | 前端运行环境 | <code>node --version</code>   | 安装 <a href="#">Node.js</a> (需 ≥20.19) |
| MySQL          | 数据库    | <code>mysql --version</code>  | 见第二步安装指引                              |

⚠ **Miniconda 用户特别注意：**Miniconda 不包含 Node.js!

如果你只安装了 Miniconda，必须单独前往 [nodejs.org](https://nodejs.org) 下载安装 Node.js，否则 `npm install` 和 `npm run dev` 将无法执行。

### 下载 Node.js®

获得适用于 Windows 且使用 Docker 和 npm 的 Node.js® v24.18.1

提示: Want new features sooner? Get the latest Node.js version instead and try the latest improvements!

```
1 # Docker 对每个操作系统都有特定的安装指导。
2 # 请参考 https://docker.com/get-started/ 给出的官方文档
3
4 # 拉取 Node.js Docker 镜像:
5 docker pull node:24-alpine
6
7 # 创建 Node.js 容器并启动一个 shell 会话:
8 docker run -it --rm --entrypoint sh node:24-alpine
9
10 # 验证 Node.js 版本:
11 node -v # Should print "v24.18.1".
12
13 # 验证 npm 版本:
14 npm -v # Should print "11.16.0".
```

PowerShell

复制到剪贴板

Docker 是一个容器化平台。如果遇到任何问题，请访问 Docker 的网站。

或者获得适用于 Windows x64 平台的 Node.js® 构建。

Windows 安装程序(.msi)

独立文件(.zip)

## 0 切换到项目根目录（重要！）

⚠ 打开终端后，默认目录是用户主目录（如 `C:\Users\lenovo` 或 `/Users/xxx`）。

必须先 `cd` 到项目根目录，否则后续所有 `cd backend` 都会报“找不到路径”！

### ① 使用 `cd` 命令进入项目目录

# 请替换为你电脑上本项目的实际路径

```
cd /path/to/cell-yolo-web
```

```
(base) C:\Users\lenovo>E:
```

```
(base) E:\>cd E:\桌面\cell-yolo-web
```

```
(base) E:\桌面\cell-yolo-web>dir
```



## ② 验证是否在正确目录

```
ls      # macOS/Linux → 应输出 README.md QUICK-START.md web
dir     # Windows → 应输出 README.md QUICK-START.md web
```

```
(base) E:\桌面\cell-yolo-web>dir
Volume in drive E is 新加卷
Volume Serial Number is E615-2EC6

Directory of E:\桌面\cell-yolo-web

2026/07/30  17:07    <DIR>          .
2026/07/29  14:38    <DIR>          ..
2026/07/29  12:32    <DIR>          .claude
2026/05/05  18:39             464 .gitignore
2026/04/05  18:20    <DIR>          .user-storage
2026/03/15  17:32    <DIR>          .vscode
2026/05/05  18:39    <DIR>          backend
2026/07/29  12:23    <DIR>          docs
2026/05/05  18:39    <DIR>          frontend
2026/03/15  17:30    <DIR>          libs
2026/03/26  12:55    <DIR>          node_modules
2026/07/30  17:13             16,071 QUICK-START.md
2026/07/29  12:44             13,719 README.md
2026/07/01  18:36             12,281 setup.ps1
2026/07/01  18:36             10,999 setup.sh
                5 File(s)              53,534 bytes
                10 Dir(s)  115,567,009,792 bytes free
```

💡 看到 README.md、QUICK-START.md 和 web 文件夹，说明目录正确。

## 1 创建并激活 Python 虚拟环境

使用 Conda 创建名为 `cell-yolo` 的虚拟环境 (推荐 Python 3.8) :

```
conda create -n cell-yolo python=3.8 -y    # 创建环境
conda activate cell-yolo                    # 激活环境
python --version                            # 验证版本
```

```
(base) E:\桌面\cell-yolo-web>conda create -n cell-yolo-example python=3.8 -y
Retrieving notices: ...working... done
Channels:
- http://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/free
- http://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/main
- defaults
Platform: win-64
Collecting package metadata (repodata.json): done
Solving environment: done
```

```
(base) E:\桌面\cell-yolo-web>conda activate cell-yolo-example
```

```
(cell-yolo-example) E:\桌面\cell-yolo-web>python --version
Python 3.8.20
```

💡 成功激活后, 终端提示符前面会出现 `(cell-yolo)` 前缀。

**注意:** 每次打开新终端启动项目前, 都需要先执行 `conda activate cell-yolo`。

## 2 安装 MySQL 数据库

### macOS

```
brew install mysql
brew services start mysql
mysql_secure_installation
```

按提示设置 root 密码即可。

### Windows

1. 下载 MySQL Installer:  
[dev.mysql.com/downloads/installer/](https://dev.mysql.com/downloads/installer/)
2. 运行安装程序, 选择 "**Developer Default**"
3. 按提示完成, 记住 root 密码
4. Win+R → `services.msc` → 找到 MySQL → 右键"启动"

### 创建数据库 (可选, 初始化脚本会自动创建)

```
mysql -u root -p
CREATE DATABASE cell_tracking CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
EXIT;
```

#### 验证 MySQL 是否启动成功:

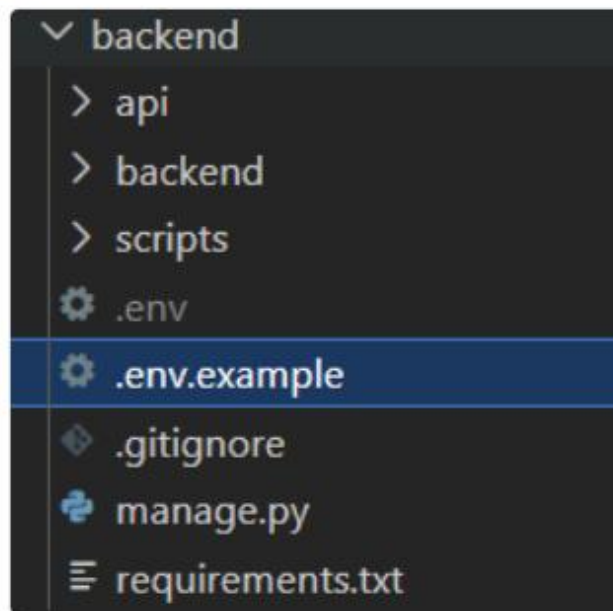
macOS: `brew services list | grep mysql` | Windows: 打开服务管理器查看 MySQL 服务状态



### 3 配置环境变量 (.env 文件)

在 `backend` 目录下创建 `.env` 文件:

```
cd backend
cp .env.example .env    # macOS/Linux
# Windows: 手动复制 .env.example 并重命名为 .env
```



编辑 `.env`, 填入以下内容 (替换密码和 API 密钥):

编辑 `.env`，填入以下内容（替换密码和 API 密钥）：

```
SECRET_KEY=django-insecure-changeme-in-production
DEBUG=True

DB_HOST=localhost
DB_PORT=3306
DB_USER=root
DB_PASSWORD=your_password      ← 替换为你的 MySQL 密码
DB_NAME=cell_tracking

DEEPSEEK_API_KEY=your_deepseek_api_key  ← 替换为你的 API 密钥
DEEPSEEK_API_BASE=https://api.deepseek.com/v1
```

⚠ **Windows 注意：**在记事本中保存时选择“所有文件”类型，避免保存为 `.env.txt`。

路径中使用正斜杠 `/` 或双反斜杠 `\\`。

## 4 安装 Python 依赖

确保在 `backend` 目录下，且已激活 `conda` 环境：

```
pip install -r requirements.txt
```

### 主要依赖一览

| 类别        | 包名                        | 版本                   | 说明            |
|-----------|---------------------------|----------------------|---------------|
| Web 框架    | Django                    | 4.2.28               | 后端框架          |
| REST API  | djangorestframework       | 3.15.2               | API 接口        |
| WebSocket | channels                  | 4.2.2                | 实时进度推送        |
| 视频处理      | opencv-python             | 4.13.0               | 视频读写          |
| 深度学习      | torch / torchvision       | 2.4.1                | 神经网络推理        |
| YOLO      | ultralytics               | 8.4.21               | 本地库，无需<br>pip |
| 追踪        | deep-sort-realtime        | 1.3.2                | 多目标跟踪         |
| 数据处理      | numpy / pandas /<br>scipy | 1.24 / 2.0 /<br>1.10 | 数据分析          |

💡 `ultralytics` 已作为本地库放在 `web/libs/ultralytics`，通过 `settings.py` 中的 `sys.path` 自动配置，无需额外安装。

## 5 初始化数据库 — 创建数据库 & Django 迁移

---

### ① 创建数据库

```
cd scripts  
python init_db.py
```

成功输出: ✓ 数据库 'cell\_tracking' 创建成功或已存在

### ② 执行 Django 迁移 (创建表结构)

```
cd ../backend  
python manage.py migrate
```

⚠ 如果报错: 通常是 MySQL 服务未启动或 .env 中密码错误。

检查方法: ① MySQL 服务是否运行 ② .env 中 DB\_PASSWORD 是否正确  
这一步报错是正常情况, 后续会进行补齐, 不要慌张。

## 5 初始化数据库 — 初始化数据

### ③ 运行数据初始化脚本

```
cd ../scripts
python init_data.py
```

脚本会自动完成：

| 步骤 | 做什么        | 说明                                              |
|----|------------|-------------------------------------------------|
| 1  | 检查数据库表结构   | 检查 users / tasks / videos 等表的字段是否完整，自动修复缺失字段    |
| 2  | 创建 root 用户 | 用户名 <code>root</code> ，密码 <code>password</code> |
| 3  | 导入默认模型     | 在 models 表中创建初始模型记录                             |

成功后关键输出：

```
✓ root 用户创建成功 (ID: 1)
  用户名: root    密码: password
✓ 导入 model 创建成功
✓ 初始化完成！
```

💡 **初始账户：** root / password — 可用于登录系统和 Django 管理后台。



## ⑥ 验证本地库（可选） + ⑦ 安装前端 npm 包

---

### 第六步：验证 YOLO 和 DeepSORT 导入

```
python -c "import ultralytics; print(f'ultralytics: {ultralytics.__version__}')"
python -c "from ultralytics import YOLO; print('✓ YOLO OK')"
python -c "from deep_sort_realtime.deepsort_tracker import DeepSORT; print('✓ DeepSORT OK')"
```

💡 导入失败？确认已 `conda activate cell-yolo`。VSCode 用户：  
`Cmd+Shift+P` → Reload Window。

## 第七步：安装前端 npm 包

```
cd ../frontend  
npm install
```

| 类别 | 包          | 版本  | 类别  | 包           | 版本  |
|----|------------|-----|-----|-------------|-----|
| 框架 | Vue 3      | 3.5 | 样式  | TailwindCSS | 4.1 |
| 构建 | Vite       | 7.3 | 路由  | Vue Router  | 5.0 |
| 语言 | TypeScript | 5.9 | 图表  | ECharts     | 5.6 |
| 状态 | Pinia      | 3.0 | 编辑器 | CodeMirror  | —   |

 **Node.js 版本要求：** ^20.19.0 或 ≥22.12.0。  
如果 `npm install` 报错，先检查 `node --version`。

## ▶ 启动前后端服务

在 `web` 目录下, 打开两个终端:

### 终端 1 — 后端

```
cd backend
conda activate cell-yolo
python manage.py runserver
```

→ <http://localhost:8000>

### 终端 2 — 前端

```
cd frontend
npm run dev
```

→ <http://localhost:5173>

```
(base) PS E:\桌面\cell-yolo-web> conda activate cell-yolo
(cell-yolo) PS E:\桌面\cell-yolo-web> cd backend
(cell-yolo) PS E:\桌面\cell-yolo-web\backend> python .\manage.py runserver
Watching for file changes with StatReloader
Performing system checks...
```

```
System check identified no issues (0 silenced).
July 31, 2026 - 17:45:46
Django version 4.2.28, using settings 'backend.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

```
(cell-yolo) PS E:\桌面\cell-yolo-web> cd frontend
(cell-yolo) PS E:\桌面\cell-yolo-web\frontend> npm run dev
```

```
> frontend@0.0.0 dev
> vite
```

```
VITE v7.3.1 ready in 1275 ms
```

```
→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ Vue DevTools: Open http://localhost:5173/__devtools__/ as a separate window
→ Vue DevTools: Press Alt(⌘)+Shift(⇧)+D in App to toggle the Vue DevTools
→ press h + enter to show help
```

## 启动故障排查

偶尔启动失败属于正常现象！大多数情况重启即可解决。

如果前端页面显示 `Request failed with status code 500` 或类似错误：

### 重启四步法

- ① 在终端 1 中按 `Ctrl+C` 停止后端
- ② 在终端 2 中按 `Ctrl+C` 停止前端
- ③ 先启动后端（`python manage.py runserver`）→ 再启动前端（`npm run dev`）
- ④ 刷新浏览器页面（`F5`）

 大部分情况下重启即可恢复正常，无需重新安装或修改任何配置。

## 其他常见问题速查

| 错误现象                       | 可能原因               | 检查方法                                                    |
|----------------------------|--------------------|---------------------------------------------------------|
| "找不到路径/文件"                 | 终端不在项目根目录          | <code>pwd</code> (macOS) 或 <code>cd</code> (Win) 查看当前路径 |
| "conda: command not found" | conda 未安装/未配置 PATH | 重新打开 Anaconda Prompt                                    |
| "npm: command not found"   | Node.js 未安装        | <code>node --version</code> 检查                          |
| 数据库连接失败                    | MySQL 服务未启动        | 检查服务管理器 / <code>brew services list</code>               |
| "ModuleNotFoundError"      | 未激活 conda 环境       | <code>conda activate cell-yolo</code>                   |
| "Access denied for user"   | .env 中密码错误         | 检查 <code>.env</code> 中 <code>DB_PASSWORD</code>         |





## 访问地址 & API 接口

启动成功后，可通过以下地址访问：

| 服务        | 地址                                           | 说明                     |
|-----------|----------------------------------------------|------------------------|
| 前端应用      | <code>http://localhost:5173</code>           | 用户界面                   |
| 后端 API    | <code>http://localhost:8000/api/test/</code> | API 测试端点               |
| Django 后台 | <code>http://localhost:8000/admin/</code>    | 管理后台 (root / password) |

### 主要 API 接口

| 方法   | 路径                                 | 说明             |
|------|------------------------------------|----------------|
| GET  | <code>/api/test/</code>            | 测试接口           |
| POST | <code>/api/upload/</code>          | 上传视频           |
| POST | <code>/api/process/</code>         | 启动处理任务         |
| GET  | <code>/api/status/:task_id/</code> | 查询任务状态         |
| GET  | <code>/api/result/:task_id/</code> | 获取处理结果         |
| GET  | <code>/api/video/:task_id/</code>  | 获取标注视频         |
| WS   | <code>/ws/task/:task_id/</code>    | WebSocket 实时进度 |



# 安装完成!

打开浏览器访问

`http://localhost:5173`

## ✦ 快速链接:

- 登录账号: `root / password`
- 项目 README: `README.md`
- 技术笔记: `docs/v2/`
- 改进路线: `docs/v2/改进方向与技术路线.md`
- 算法答辩准备: `docs/v2/具体要求.md`